

SDR-SLC

SLC-CP Control Plane Specification

Field	Value
Document	SDR-SLC-CP-1.0
Title	SLC-CP Control Plane Specification
Version	Protocol 1.0 Release 1.1 (revised for implementers)
Status	RELEASE CANDIDATE
Date	May 2026
Author	Ivo van Ling, PA2IX
Organisation	IXChange BV
Licence	Creative Commons CC BY 4.0 — Open Source

Table of Contents

Table of Contents	2
Revision History.....	5
1. Introduction.....	6
2. Normative Language	7
3. Implementation Profiles	8
4. Building a Minimal SDR-SLC Device.....	10
4.1 What a Minimal RX Implementation Looks Like	10
4.2 Adding Minimal TX to Tier 1.....	11
4.3 What a Minimal Device May Omit.....	12
4.4 How a Tier 1 Device Interoperates with a Full Client.....	13
4.5 Startup Sequence for Any Profile.....	13
5. Terminology.....	15
6. Transport and Framing	16
6.1 TCP Connection.....	16
6.2 Message Encoding	16
6.3 Length-Prefixed Framing	16
6.4 Important Field Conventions	16
7. Message Model	17
7.1 Request.....	17
7.2 Successful Response	17
7.3 Error Response.....	17
7.4 Asynchronous Event.....	17
8. Version Negotiation [T1-RX].....	19
8.1 hello	19
8.2 ping [T2].....	20
9. Capability Exchange [T1-RX]	21
9.1 get_capabilities	21
Purpose	21
Tier 1 RX Minimum Response	21
Tier 2 Additional Fields in capabilities.rx	21
Tier 1 RX/TX: Adding TX to capabilities	21
Control Descriptor Type System	22
Standard Control Names.....	22
10. Status Reporting [T2].....	24
10.1 get_status	24
10.2 hardware_monitors Object.....	25
11. Configuration Commands.....	27

11.1 set_frequency [T1-RX]	27
11.2 set_sample_rate [T1-RX]	28
11.3 set_control [T1-RX for rf_gain · T2 for all others]	29
11.4 get_control [T2]	30
12. Stream Session Management [T1-RX]	32
12.1 open_stream (RX) [T1-RX]	32
12.2 open_stream (TX) [T1-RXTX]	34
12.3 start_rx [T1-RX]	35
12.4 stop_rx [T1-RX]	36
12.5 start_tx [T1-RXTX]	37
12.6 stop_tx [T1-RXTX]	37
12.7 close_stream [T1-RX]	38
12.8 PTT and Turnaround Mode	39
manual turnaround_mode	39
auto turnaround_mode	39
13. Transmitter Control Commands	41
13.1 get_tx_capabilities [T1-RXTX]	41
13.2 set_tx_power [T2, optional at T1-RXTX]	42
13.3 get_vswr [T3]	42
13.4 auto_tune [T3]	43
14. Reference Level and Power Display [T2]	45
15. Clock and Location [T2 query · T3 set]	46
15.1 Clock Sources	46
15.2 get_clock_status [T2]	46
15.3 Location [T3]	46
16. Asynchronous Events	47
16.1 adc_overload Event	47
16.2 monitor_alert Event	48
17. Error Code Registry	49
18. Extensibility and Forward Compatibility	50
18.1 Reserved Field Names	50
19. Conformance Requirements	52
19.1 Tier 1 Minimal RX Server	52
19.2 Tier 1 Minimal RX/TX Server	52
19.3 Tier 2 Standard Server	53
19.4 Tier 3 Full Server	53
19.5 Client Requirements (any tier)	54
20. Security Considerations	55
21. References	56

21.1 Normative References	56
21.2 Informative References.....	56

Revision History

Rev	Date	Author	Description
1.0 draft	2026-05-01	I. van Ling	Initial draft. Core command set established.
1.1 draft	2026-05-16	I. van Ling	TX commands, turnaround_mode, conformance tiers.
1.2 draft	2026-05-20	I. van Ling	Typed control system, AGC, bias_tee, PPM, direct_sampling, preferred hints.
1.3 draft	2026-05-21	I. van Ling	signal_type in open_stream. VITA-49 Context packet client behaviour.
1.4 draft	2026-05-22	I. van Ling	Reference Level calibration in capabilities, get_status, set_control trigger rule.
1.5 draft	2026-05-22	I. van Ling	adc_overload, adc_level_dbfs, lo_locked, actual_sample_rate_sps, hardware_monitors, monitor_alert event, bias_tee float redesign, reserved field names.
1.0 release	2026-05-22	I. van Ling	Consolidated release. Three-tier structure.
1.1 release	2026-05-23	I. van Ling	Revised for implementers. Four implementation profiles (Tier 1 RX, Tier 1 RX/TX, Tier 2, Tier 3). Restored implementation-guide narrative and full command detail. Added §4 'Building a Minimal SDR-SLC Device'. TX no longer requires VSWR, GPS, or hardware monitors. Hobbyist-focused writing throughout.

1. Introduction

The SDR-SLC project exists because modern networked software-defined radio is technically mature but practically inaccessible. The equipment that does it well is expensive, runs on a single operating system, and speaks proprietary protocols. This project aims to fix all three of those problems.

SLC-CP is the control plane for SDR-SLC hardware. It is a JSON protocol over TCP. JSON was chosen deliberately: it is human-readable, you can debug it with netcat, and every programming language from MicroPython to Swift can parse it. The control plane tells the radio what frequency to tune, what sample rate to use, and when to start and stop streaming. Everything else — the DSP, the waterfall, the audio — runs on the host computer.

The hardware side of SDR-SLC is designed for home-builders. The reference design uses an STM32H7 microcontroller and a Tayloe quadrature detector for the RF front end, streams I/Q samples over 100BASE-T Ethernet, and can be built for under €100 in components. An RTL-SDR dongle bridged to this protocol via a Raspberry Pi daemon is also a fully conformant implementation at Tier 1.

The protocol is intentionally tiered. You do not have to implement everything to be compatible. A Tier 1 RX implementation is nine commands and maybe takes a weekend to implement. A Tier 1 RX/TX implementation adds seven more commands and is still achievable on a microcontroller. More capable hardware adds more features without breaking simpler clients.

This document is both a normative specification and a practical implementation handbook. Every command section explains what the command does, why it exists, what you must implement, and how to implement it on constrained hardware. Read the whole document once, then use it as a reference.

NOTE

The SDR-SLC protocol suite consists of four documents. This document covers the control plane only.

SDR-SLC-PS-1.0 — Protocol Suite Architecture (read this first for context).

SDR-SLC-DISC-1.0 — Service Discovery (mDNS/DNS-SD, how clients find hardware).

SDR-SLC-VITA-1.0 — I/Q Stream Transport (VITA-49 packet format and calibration).

2. Normative Language

The following keywords appear throughout this document and have precise meanings defined by RFC 2119:

Keyword	Meaning
SHALL	Absolute requirement. An implementation that omits or contradicts this is non-conformant.
SHALL NOT	Absolute prohibition.
SHOULD	Strong recommendation. An implementation may deviate but must have a good reason for doing so.
SHOULD NOT	Strong recommendation against.
MAY	Optional. A conformant implementation may include or omit this feature freely.

Implementation profile markers appear in command headings throughout this document. They indicate the minimum profile at which a command or field is required:

Marker	Meaning
[T1-RX]	Required for Tier 1 Minimal RX (and all higher profiles).
[T1-RXTX]	Required for Tier 1 Minimal RX/TX (and all higher profiles). Not required for RX-only.
[T2]	Required for Tier 2 Standard (and Tier 3). Optional for Tier 1.
[T3]	Required for Tier 3 Full only. Optional at all lower profiles.
[ANY]	May be present at any tier. Never required, but always permitted.

3. Implementation Profiles

SDR-SLC devices are classified into four implementation profiles. A device claims a profile by advertising it in the SLC-DISC TXT record and by implementing the commands listed for that profile. A Tier 2 device implements everything Tier 1 requires, plus the Tier 2 additions. A Tier 3 device implements everything.

The key design decision in the profile model is that TX capability does NOT automatically require advanced features. A home-built HF transceiver with basic PTT functionality is Tier 1 RX/TX. It does not need VSWR monitoring, GPS, PTP clocking, hardware monitors, or auto-tuning. Those are Tier 3 features.

Tier 1: Minimal RX — weekend implementable, RX only

Tier 1 Minimal RX is the entry point. Nine commands. Samples flow. Nothing else is required. Target platforms: RTL-SDR daemon on Raspberry Pi, bare-metal STM32, MicroPython on ESP32 with SPI ADC. A competent developer can reach Tier 1 RX in a weekend.

Required commands: `hello`, `get_capabilities` (minimal), `set_frequency`, `set_sample_rate`, `set_control` (rf_gain only), `open_stream` (RX), `start_rx`, `stop_rx`, `close_stream`.

NOTE

A Tier 1 RX device returns `unknown_command` for anything not implemented. It does not need stub handlers. Clients that receive `unknown_command` know the feature is not available.

Tier 1: Minimal RX/TX — weekend implementable, transceiver

Tier 1 Minimal RX/TX adds basic transmit capability to the Tier 1 RX baseline. The additional commands cover TX frequency, sample rate, stream management, and PTT. This profile is designed for the home-built HF transceiver: a Tayloe front end, an STM32 for I/Q streaming, and PTT switching. VSWR monitoring, auto-tuning, GPS, and hardware monitors are all optional.

Additional required commands (beyond Tier 1 RX): `get_tx_capabilities` (minimal), `set_frequency` (TX), `set_sample_rate` (TX), `open_stream` (TX), `start_tx`, `stop_tx`, `close_stream` (TX). Optional: `set_tx_power`.

Tier 2: Standard — full-featured daemon or firmware

Tier 2 Standard is the target for a well-engineered RTL-SDR daemon intended for distribution, or a more capable hardware implementation. It adds the full standard control set (AGC, bias tee, PPM correction, direct sampling), Reference Level power calibration, the VITA-49 Context packet, ADC monitoring, and the complete asynchronous event model. A Tier 2 device enables the full SDR-SLC Swift client feature set on macOS.

Tier 3: Full — advanced hardware with telemetry

Tier 3 Full is the target for the IXChange SDR transceiver hardware and any serious implementation that includes VSWR monitoring, antenna tuning, GPS, PTP clocking, and hardware health monitoring. All protocol features are required.

The following table summarises which capability areas apply to each profile:

Capability area	T1 RX	T1 RX/TX	T2	T3
Core RX commands (9)	SHALL	SHALL	SHALL	SHALL
Basic TX commands (7)	Omit	SHALL	SHALL	SHALL
set_tx_power	Omit	SHOULD	SHALL	SHALL
ping	MAY	MAY	SHALL	SHALL
get_status	MAY	MAY	SHALL	SHALL
get_clock_status	MAY	MAY	SHOULD	SHALL
Standard controls (AGC, bias_tee, PPM ...)	MAY	MAY	SHALL	SHALL
Reference Level calibration	MAY	MAY	SHALL	SHALL
VITA-49 Context packet emission	MAY	MAY	SHALL	SHALL
ADC monitoring (adc_overload, adc_level)	MAY	MAY	SHOULD	SHALL
LO lock status (lo_locked)	MAY	MAY	SHOULD	SHALL
Asynchronous event model (beyond stream_state)	MAY	MAY	SHALL	SHALL
hardware_monitors object	Omit	Omit	SHALL	SHALL
VSWR monitoring (get_vswr)	Omit	Omit	Omit	SHALL
auto_tune	Omit	Omit	Omit	SHALL
hardware_monitor_thresholds	Omit	Omit	MAY	SHOULD
GPS location (get_location / set_location)	Omit	Omit	Omit	SHALL
PTP clock (get_clock_status full)	MAY	MAY	SHOULD	SHALL

NOTE

"Omit" means the capability is not required and the associated commands, fields, and events MAY be entirely absent. A device that receives a command it does not implement SHALL return unknown_command. "MAY" means the device may implement the feature voluntarily without claiming a higher profile.

4. Building a Minimal SDR-SLC Device

This chapter is a practical guide for the experimenter who wants to get samples flowing as quickly as possible. It explains exactly what you need to implement, in what order, and what you can safely skip for now.

4.1 What a Minimal RX Implementation Looks Like

A Tier 1 RX device implements exactly nine commands. When a client connects, the conversation looks like this:

```
// 1. Client connects on TCP port 4620.
// 2. Client sends hello.
C→S { "type":"request", "id":"h-1", "command":"hello",
      "params":{"client_name":"My SDR Client", "client_version":"1.0",
                "supported_protocol_versions":["1.0"]} }
S→C { "type":"response", "id":"h-1", "status":"ok",
      "result":{"protocol_name":"SDR-SLC Control Plane",
                "selected_protocol_version":"1.0",
                "device_name":"My RTL-SDR Bridge",
                "manufacturer":"PA2IX" } }

// 3. Client reads capabilities.
C→S { "type":"request", "id":"cap-1", "command":"get_capabilities", "params":{} }
S→C { "type":"response", "id":"cap-1", "status":"ok",
      "result":{"capabilities":{"schema":"sdrslc.capabilities", "schema_version":1,
                              "full_duplex":false,
                              "rx":{"supported":true,
                                    "frequency_ranges_hz":[{"min":100000, "max":1766000000}],
                                    "sample_rates_sps":[960000, 1024000, 1536000, 2048000, 2400000],
                                    "vita_port":4621,
                                    "samples_per_packet":360,
                                    "iq_formats":[{"encoding":"unsigned", "bits_per_component":8,
                                                  "container_bits":8, "packing":"interleaved",
                                                  "byte_order":"big_endian", "native":true,
                                                  "supported":true, "label":"u8"}]
                                },
                              "controls":[{"name":"rf_gain", "type":"float", "unit":"dB",
                                           "min":0, "max":49.6, "step":0.1, "default":20 }
                                ]
                              }
      }
    }

// 4. Client tunes to 14.1 MHz.
C→S { "type":"request", "id":"f-1", "command":"set frequency",
      "params":{"direction":"rx", "frequency_hz":14100000 } }
S→C { "type":"response", "id":"f-1", "status":"ok",
      "result":{"direction":"rx", "applied_frequency_hz":14100000 } }

// 5. Client sets sample rate.
C→S { "type":"request", "id":"sr-1", "command":"set_sample_rate",
      "params":{"direction":"rx", "sample_rate_sps":2048000 } }
S→C { "type":"response", "id":"sr-1", "status":"ok",
      "result":{"direction":"rx", "applied_sample_rate_sps":2048000 } }

// 6. Client sets gain.
C→S { "type":"request", "id":"g-1", "command":"set control",
      "params":{"direction":"rx", "name":"rf_gain", "value":20.0 } }
S→C { "type":"response", "id":"g-1", "status":"ok",
      "result":{"direction":"rx", "name":"rf_gain", "applied_value":20.0 } }

// 7. Client opens a VITA-49 stream.
C→S { "type":"request", "id":"os-1", "command":"open_stream",
      "params":{"direction":"rx", "protocol":"vita49",
                "protocol_version":"VITA-A/1.0",
                "format":{"encoding":"unsigned", "bits_per_component":8,
                          "container_bits":8, "packing":"interleaved",
                          "byte_order":"big_endian" },
                "destination":{"ip":"192.168.1.20", "port":60000 } } }
S→C { "type":"response", "id":"os-1", "status":"ok",
```

```

        "result":{ "stream_id":"rx-1", "direction":"rx",
                    "source":{"ip":"192.168.5.35", "port":4621 },
                    "destination":{"ip":"192.168.1.20", "port":60000 },
                    "sample_rate_sps":2048000, "samples_per_packet":360,
                    "format":{"encoding":"unsigned","bits_per_component":8,
                               "container_bits":8,"packing":"interleaved",
                               "byte_order":"big_endian","native":true,
                               "supported":true,"label":"u8" },
                    "vita":{"stream_id":1, "packet_class":"if data",
                             "timestamp_mode":"integer_and_fractional" } } }

// 8. Client starts the stream.
C→S { "type":"request", "id":"rx-1", "command":"start_rx",
      "params":{"stream_id":"rx-1" } }
S→C { "type":"response", "id":"rx-1", "status":"ok",
      "result":{"direction":"rx", "streaming":true } }
// Device emits: { "type":"event","event":"stream_state",
//   "params":{"stream_id":"rx-1","streaming":true,"direction":"rx" } }

// --- UDP VITA-49 packets now flow to 192.168.1.20:60000 ---

// 9. Shutdown.
C→S { "type":"request","id":"rx-2","command":"stop_rx",
      "params":{"stream_id":"rx-1" } }
C→S { "type":"request","id":"cs-1","command":"close_stream",
      "params":{"stream_id":"rx-1" } }
// Client closes TCP connection.

```

TIP

If you are building an RTL-SDR bridge daemon, start with just these nine commands. Get them working and tested end-to-end before adding anything else. The client will tell you what it needs by returning error messages for features you have not implemented yet.

4.2 Adding Minimal TX to Tier 1

A Tier 1 RX/TX device adds seven commands to the Tier 1 RX baseline. The client opens a TX stream, sends I/Q samples to the device over UDP, and the device outputs them to the DAC and PA. The PTT sequence depends on the `turnaround_mode` reported in capabilities.

A simple hand-built transceiver reports `turnaround_mode`: "manual". This means the client explicitly stops RX before starting TX, and restarts RX after TX ends. The device does not need to manage this internally.

```

// Minimal TX sequence (manual turnaround mode).

// Step 1: Advertise TX in get_capabilities.
//   "full_duplex": false,
//   "turnaround_mode": "manual",
//   "tx": {
//     "supported": true,
//     "frequency_ranges_hz": [{ "min":1800000, "max":30000000 }],
//     "sample_rates_sps": [48000, 96000],
//     "vita port": 4622,
//     "samples_per_packet": 360,
//     "iq_formats": [ ... ]
//   }

// Step 2: Get TX capabilities (client calls this before TX).
C→S { "type":"request","id":"tc-1","command":"get tx capabilities","params":{} }
S→C { "type":"response","id":"tc-1","status":"ok","result":{
      "supported":true,
      "frequency_ranges_hz":[{ "min":1800000, "max":30000000 }],
      "sample_rates_sps":[48000,96000],
      "iq_formats":[{"encoding":"twos complement","bits_per_component":16,
                       "container_bits":16,"packing":"interleaved",
                       "byte_order":"big_endian","native":true,
                       "supported":true,"label":"int16" }],
      "vita port":4622, "samples_per_packet":360,
      "vswr_monitor":false, "auto_tune":false } }

```

```
// Step 3: Configure TX frequency and rate.
C→S set_frequency(direction=tx, frequency_hz=14200000)
C→S set_sample_rate(direction=tx, sample_rate_sps=48000)

// Step 4: Open TX stream.
C→S { "type":"request","id":"os-2","command":"open stream",
      "params":{"direction":"tx", "protocol":"vita49",
                "protocol version":"VITA-A/1.0",
                "sample_rate_sps":48000,
                "format":{"encoding":"twos complement",
                          "bits_per_component":16,"container_bits":16,
                          "packing":"interleaved","byte order":"big endian" },
                "source":{"ip":"192.168.1.20", "port":60001 } } }
S→C { "type":"response","id":"os-2","status":"ok",
      "result":{"stream_id":"tx-1","direction":"tx",
                "source":{"ip":"192.168.5.35","port":4622 },
                "destination":{"ip":"192.168.1.20","port":60001 } } }

// Step 5: PTT press (manual turnaround).
C→S stop_rx(stream_id=rx-1) // pause waterfall
C→S start_tx(stream_id=tx-1) // begin transmit
// Client now sends VITA-49 UDP to device port 4622.

// Step 6: PTT release.
C→S stop_tx(stream_id=tx-1)
C→S start_rx(stream_id=rx-1) // resume waterfall
```

TIP

For a minimal TX implementation you do not need set_tx_power, get_vswr, or auto_tune. Return not_supported for all three. The client will display the power control as unavailable and skip the VSWR display.

4.3 What a Minimal Device May Omit

The following table lists all features that a Tier 1 device (both RX and RX/TX) may omit entirely. The client will handle the absence gracefully as long as you return unknown_command for unimplemented commands and not_supported for direction=tx on a pure RX device.

Feature	What to return when asked
ping	unknown_command
get_status	unknown_command
get_clock_status	unknown_command
set_clock / get_location / set_location	unknown_command
AGC control (agc)	not_supported (via set_control)
Bias tee (bias_tee)	not_supported (via set_control)
PPM correction (ppm_correction)	not_supported (via set_control)
Direct sampling (direct_sampling)	not_supported (via set_control)
Reference Level fields in get_capabilities	Simply omit from response
hardware_monitors in get_status	Omit get_status or omit the object
VITA-49 Context packets	Simply do not send them; client assumes complex IQ

Feature	What to return when asked
VSWR / auto_tune (T1 RX/TX)	not_supported
hardware_monitor_thresholds	Omit from capabilities
Asynchronous events except stream_state	Simply do not emit them

NOTE	<i>The stream_state event (sent on start_rx / stop_rx) is the one event that IS expected even at Tier 1. A Tier 1 device should emit it. Everything else can be omitted.</i>
-------------	--

4.4 How a Tier 1 Device Interoperates with a Full Client

The SDR-SLC Swift client is a Tier 3 client. It queries get_capabilities at startup and adapts its feature display based on what the device reports. Here is what happens when a Tier 3 client connects to a Tier 1 RX device:

- The client reads capabilities and sees no preferred_sample_rate_sps. It uses the first entry in sample_rates_sps instead.
- The client reads capabilities and sees no reference_level_dbm_at_preferred_gain. The power display shows 'uncalibrated' rather than a dBm value.
- The client calls ping. The device returns unknown_command. The client disables its connection health monitor.
- The client calls get_status. The device returns unknown_command. The client disables the ADC level meter and LO lock indicator.
- The client calls set_control for agc. The device returns not_supported. The client greys out the AGC checkbox.
- No Context packets arrive. The client assumes complex IQ mode and proceeds normally.
- TX controls in the UI are greyed out because capabilities.tx.supported is false.

The client degrades gracefully. The operator gets a functional waterfall and receiver with manual gain control. Everything works. Adding Tier 2 features later does not require a client update — the client already knows how to use them.

4.5 Startup Sequence for Any Profile

Regardless of profile, the startup sequence is always the same. The device must be ready to accept commands in this order:

1. TCP connection accepted on port 4620.
2. hello received and validated. Protocol version selected and returned.
3. get_capabilities received. Full capabilities returned.
4. Configuration commands (set_frequency, set_sample_rate, set_control). Applied in any order.
5. open_stream received. Stream session created. stream_id returned.
6. start_rx (or start_tx) received. Streaming begins. stream_state event emitted.
7. Stream runs. Events emitted as conditions change.
8. stop_rx (or stop_tx) received. Streaming stops. stream_state event emitted.
9. close_stream received. Session released.
10. TCP connection closed. Device tears down any remaining sessions.

TIP

At Tier 1, you do not need to handle steps 4 and 7 specially. You can process commands in any order and emit `stream_state` as the only event. The client handles everything else.

5. Terminology

Term	Definition
SDR-SLC Client	Host software (macOS, Linux, Windows) that opens a SLC-CP control connection.
SDR-SLC Device	Radio hardware or software bridge that acts as the SLC-CP server.
Direction	Either rx (receive) or tx (transmit). Commands that affect one path carry a direction field.
Request	A client-initiated message with a command field and optional params.
Response	A device-generated reply. Carries the same id as the request and a status of ok or error.
Event	An asynchronous device-generated message. Not correlated to any request.
id	A client-generated string that uniquely identifies a request within the session. The device echoes it in the response.
Stream ID	An opaque string assigned by the device to identify an open stream session.
Control descriptor	A JSON object in capabilities.[direction].controls[] that describes one settable hardware parameter: its name, type, range, and default value.
Reference Level	The power level at the antenna port corresponding to a full-scale I/Q sample, in dBm. Used to convert sample amplitude to absolute received power.
dBFS	Decibels relative to full scale. 0 dBFS is the maximum representable sample amplitude.
session	The period from TCP connection to TCP disconnection. All stream state is scoped to a session.
turnaround	The transition from RX to TX or TX to RX in a half-duplex system.

6. Transport and Framing

6.1 TCP Connection

SLC-CP runs over TCP/IP. The device SHALL [T1-RX] listen on TCP port 4620 by default. This port is advertised in the SLC-DISC mDNS TXT record, so clients learn it automatically without configuration. The device accepts any number of simultaneous client connections; behaviour when multiple clients issue conflicting commands is implementation-defined.

TIP

On an STM32 with lwIP, one TCP PCB on port 4620 with a receive callback that dispatches to your command handler is sufficient. You do not need a thread per connection for a basic implementation.

6.2 Message Encoding

Every message is a single UTF-8 encoded JSON object. Multi-byte Unicode characters are permitted in string values. Control characters (0x00–1F) SHALL NOT appear unescaped inside JSON strings. The device must accept messages up to 65,536 bytes in payload length. Longer messages may be rejected by closing the connection.

6.3 Length-Prefixed Framing

Each message is framed with a 4-byte big-endian unsigned integer that gives the byte length of the JSON payload, followed immediately by the payload:

```
Frame = [ length_prefix : uint32, 4 bytes, big-endian ]  
        [ payload       : UTF-8 JSON, exactly length_prefix bytes ]
```

The `length_prefix` covers the payload bytes only, not itself.
A frame with `length_prefix=47` is $4 + 47 = 51$ bytes on the wire.

TIP

In C on an STM32 or lwIP: receive 4 bytes, `ntohl()` to get the payload length, then receive exactly that many more bytes. Pass the payload to your JSON parser. cJSON is small enough to run on an STM32H7.

6.4 Important Field Conventions

Two field types need special handling on 32-bit systems:

- All fields ending in `_hz` are 64-bit unsigned integers. On STM32 or 32-bit Linux, use `uint64_t`. On JavaScript clients, use `BigInt`. A 32-bit integer can only represent up to ~4.3 GHz, which is fine today but will break for UHF/SHF hardware.
- Fields named `epoch_s` are 64-bit signed integers (seconds since the Unix epoch). Same treatment applies.

7. Message Model

There are three message types in SLC-CP: request (client to device), response (device to client), and event (device to client, unsolicited). Every message is a JSON object with a type field that identifies which kind it is.

7.1 Request

```
{
  "type"    : "request",
  "id"      : "any-unique-string-you-choose",
  "command" : "command name",
  "params"  : { /* command-specific parameters */ }
}
```

Field	Type	Required	Description
type	string	YES	Literal: "request".
id	string	YES	Client-chosen identifier. Echoed in the response. Must be unique within the session. A counter or UUID both work.
command	string	YES	Command name. See individual command definitions.
params	object	COND	Command parameters. May be omitted for commands that take no parameters, but {} is also acceptable.

7.2 Successful Response

```
{
  "type"    : "response",
  "id"      : "echoed-from-request",
  "status"   : "ok",
  "result"   : { /* command-specific result fields */ }
}
```

7.3 Error Response

```
{
  "type"    : "response",
  "id"      : "echoed-from-request",
  "status"   : "error",
  "error"    : {
    "code"    : "error code string",
    "message" : "Human-readable explanation for logging or display",
    "details" : { /* optional structured context */ }
  }
}
```

The error.code field is a machine-readable string from the error code registry in §16. The error.message field is for humans. The optional error.details object may carry structured context specific to the error code.

7.4 Asynchronous Event

```
{
  "type"    : "event",
  "event"    : "event name",
  "params"   : { /* event-specific parameters */ }
}
```

Events carry no id field. They are emitted whenever a device condition occurs, independent of any request. A Tier 1 device only needs to emit stream_state. Clients SHALL silently ignore events they do not recognise.

NOTE

Unknown fields in any message SHALL be silently ignored. This is the fundamental extensibility rule of SLC-CP. If a future client adds a field your device does not understand, your device must still process the message correctly without that field.

8. Version Negotiation [T1-RX]

8.1 hello

Required at: all profiles

Purpose

Establish the protocol version and exchange device identity. hello SHALL be the first command sent by the client after connecting. The device SHALL NOT process any other command until hello has been successfully completed. If a client sends any command before hello, the device SHALL return an invalid_request error.

Request Fields

Field	Type	Required	Description
client_name	string	YES	Human-readable name of the client application. e.g. "SDR-SLC Swift Client".
client_version	string	YES	Client software version string. Informational only.
supported_protocol_versions	string array	YES	List of protocol versions the client supports. e.g. ["1.0"]. The device selects the highest mutually supported version.

Response Fields (result object)

Field	Type	Description
protocol_name	string	Always "SDR-SLC Control Plane".
selected_protocol_version	string	The version the device has selected from the client's list.
device_name	string	Human-readable name of the hardware device.
manufacturer	string	Manufacturer name. IXChange hardware: "IXChange BV".

Behaviour

- The device SHALL select the highest version in the client's supported_protocol_versions list that the device also supports.
- If no version is mutually supported, the device SHALL return unsupported_protocol_version and MAY close the TCP connection.
- The device MAY include additional informational fields (hardware revision, firmware version) in the result object. Clients SHALL ignore unknown fields.

Error Codes

Error code	Condition
unsupported_protocol_version	No version in the client's list is supported by the device.
invalid_request	hello received after session was already established, or malformed envelope.

Example

```
// Request
{ "type":"request", "id":"h-1", "command":"hello",
  "params":{"client_name":"My SDR Client", "client_version":"1.0.0",
    "supported_protocol_versions":["1.0"]} }

// Response
{ "type":"response", "id":"h-1", "status":"ok",
  "result":{"protocol_name":"SDR-SLC Control Plane",
    "selected_protocol_version":"1.0",
    "device_name":"SDR-SLC HF Transceiver V1.0",
    "manufacturer":"IXChange BV" } }
```

TIP

On an STM32, set a boolean flag after a successful hello response. Check this flag at the start of every other command handler. If false, return `invalid_request`. This is the only session guard you need at Tier 1.

8.2 ping [T2]

Required at: Tier 2 and above | Optional at: Tier 1

Purpose

Test whether the control connection is alive and measure round-trip latency. ping carries no state change and has no side effects. The device simply responds immediately.

Behaviour

- The device SHALL respond as quickly as possible, without waiting for any hardware operation.
- ping may be sent at any time after hello. It does not affect stream state.

Example

```
{ "type":"request", "id":"p-1", "command":"ping", "params":{} }
{ "type":"response", "id":"p-1", "status":"ok", "result":{} }
```

TIP

At Tier 1, return `unknown_command`. At Tier 2, implement ping as a one-liner: receive the request, send ok with an empty result.

9. Capability Exchange [T1-RX]

`get_capabilities` is the second command the client calls after `hello`. It is the foundation of the entire session: the client reads the capabilities and adapts everything else — its UI controls, its default settings, its knowledge of what commands are available — based on what the device reports.

A Tier 1 device must return a minimal but correct capabilities response. A Tier 3 device returns a rich response. The client handles everything in between gracefully.

9.1 `get_capabilities`

Purpose

Return the complete capability set of the device. The client SHALL call this before any radio configuration command.

Tier 1 RX Minimum Response

The following is the minimum capabilities response a Tier 1 RX device must return. Every field shown is required. Fields marked optional may be added at any time without breaking the protocol.

```
{ "type": "response", "id": "cap-1", "status": "ok",
  "result": { "capabilities": {
    "schema"       : "sdrslc.capabilities",
    "schema_version" : 1,
    "full_duplex"   : false,
    "rx": {
      "supported"      : true,
      "frequency_ranges_hz" : [{ "min": 100000, "max": 1766000000 }],
      "sample_rates_sps" : [960000, 1024000, 1536000, 2048000],
      "vita_port"       : 4621,
      "samples_per_packet" : 360,
      "iq_formats": [
        { "encoding": "unsigned", "bits_per_component": 8, "container_bits": 8,
          "packing": "interleaved", "byte_order": "big endian",
          "native": true, "supported": true, "label": "u8" }
      ],
      "controls": [
        { "name": "rf_gain", "type": "float", "unit": "dB",
          "min": 0, "max": 49.6, "step": 0.1, "default": 20 }
      ]
    }
  }
}
```

Tier 2 Additional Fields in `capabilities.rx`

Field	Tier	Description
<code>preferred_sample_rate_sps</code>	[T2]	The rate at which the hardware performs optimally. The client uses this as its default. RTL-SDR: typically 2048000. IXChange SDR: 1024000.
<code>preferred_gain_db</code>	[T2]	Suggested starting gain for typical reception. The client pre-populates the gain control with this value.
<code>reference_level_dbm_at_preferred_gain</code>	[T2]	Power at the antenna port that corresponds to a full-scale I/Q sample when gain is at <code>preferred_gain_db</code> . Used by the client to display received power in dBm. See §14.

Tier 1 RX/TX: Adding TX to capabilities

A Tier 1 RX/TX device adds a `tx` sub-object to capabilities alongside the `rx` sub-object. The `tx` sub-object follows the same structure as `rx`. The `full_duplex` field indicates whether simultaneous RX and

TX is supported (most simple implementations: false). The `turnaround_mode` field (required for RX/TX) tells the client how PTT transitions are handled.

```
// Additional fields at the top level for RX/TX devices:
"full_duplex"      : false,
"turnaround_mode" : "manual", // or "auto" if device handles RX muting

// TX sub-object (minimal):
"tx": {
  "supported"      : true,
  "frequency_ranges_hz" : [{ "min":1800000, "max":30000000 }],
  "sample_rates_sps"  : [48000, 96000, 192000],
  "vita_port"         : 4622,
  "samples_per_packet" : 360,
  "iq_formats": [
    { "encoding":"twos complement", "bits per component":16, "container_bits":16,
      "packing":"interleaved", "byte_order":"big_endian",
      "native":true, "supported":true, "label":"int16" }
  ]
}

// If TX is not supported, still include the tx object with supported:false.
// This tells the client definitively that TX is unavailable.
"tx": { "supported":false, "channels":0 }
```

NOTE A Tier 1 RX device does not need to include the `tx` sub-object at all. A Tier 2 or higher device SHALL include it, even when TX is not supported, with `supported:false`. This allows clients to check `capabilities.tx.supported` without guarding against the field being absent.

Control Descriptor Type System

Every control in the `controls[]` array has a `type` field that tells the client what kind of value to send in `set_control`. The type also determines which additional fields the descriptor carries:

type	Additional fields	set_control value type
"float"	min, max, step, unit (optional), default, allowed_values (optional)	JSON number. E.g. 24.0
"integer"	min, max, step, default	JSON integer. E.g. -14
"boolean"	default	JSON true or false
"enum"	values (string array), default	JSON string from values array. E.g. "i_branch"

TIP At Tier 1 you only need one control: `rf_gain` as a float. The `type` field on `rf_gain` is required. Clients use the `type` field to decide what kind of UI widget to display and what value type to send back.

Standard Control Names

The following control names are reserved and have defined meanings. Use these names for these hardware functions. Vendor-specific controls should use a prefixed name such as `"myco.custom_filter"`.

name	type	Tier	Description
<code>rf_gain</code>	float	[T1-RX]	Combined RF/IF gain in dB. The primary gain control. Minimum requirement: every device must expose this control.

name	type	Tier	Description
agc	boolean	[T2]	Hardware automatic gain control. When true, the hardware manages rf_gain autonomously and rf_gain set_control commands are ignored.
bias_tee	float	[T2]	Bias tee voltage in volts. 0.0 = off. For RTL-SDR V3/V4 use min=0, max=5, step=5, allowed_values=[0.0, 5.0]. For LNB driver hardware use allowed_values=[0.0, 13.0, 18.0].
ppm_correction	integer	[T2]	Frequency correction in parts per million to compensate for oscillator error. Negative = shift centre frequency down.
direct_sampling	enum	[T2]	RTL-SDR HF direct sampling mode. Values: "off", "i_branch", "q_branch". Bypasses the tuner and connects the antenna to one ADC input.
lna_gain	float	[T3]	Separate LNA stage gain where the hardware exposes it independently from the IF stage.
if_gain	float	[T3]	IF gain stage, separate from LNA.

10. Status Reporting [T2]

10.1 get_status

Required at: Tier 2 and above | Optional at Tier 1 (return unknown_command)

Purpose

Query the current operational state of the device, including stream activity, current frequency and sample rate, reference level, ADC health, and all applied control values. The client uses this for its status display, signal level meter, and gain indicators.

Response Fields (result object)

Field	Type	Description
rx.enabled	boolean	true if RX hardware is present and initialised.
rx.streaming	boolean	true if an RX stream is currently active.
rx.frequency_hz	integer	Current centre frequency in Hz.
rx.sample_rate_sps	integer	Configured sample rate in SPS.
rx.actual_sample_rate_sps	integer/null	[T2] Actual hardware rate. May differ from sample_rate_sps due to clock divider constraints. The client uses this for waterfall frequency scaling. null if the device cannot determine it independently.
rx.reference_level_dbm	number/null	[T2] Current Reference Level in dBm. Tracks gain changes. Consistent with the most recently sent VITA-49 Context packet. null if uncalibrated.
rx.adc_overload	boolean/null	[T2] true if ADC is currently clipping. null if hardware cannot detect this.
rx.adc_level_dbfs	number/null	[T2] RMS ADC input level in dBFS (0.0 = full scale). null if not measurable.
rx.lo_locked	boolean/null	[T2] true if the local oscillator PLL is locked. null if the hardware does not expose lock status.
rx.controls{}	object	[T2] Current applied value of every control. Keys are control names; values match the control's type.
rx.hardware_monitors{}	object	[T2] Named sensor channels. All three standard channels present; null for any sensor not fitted. See §10.2.
tx.enabled	boolean	true if TX hardware is present.
tx.streaming	boolean	true if a TX stream is currently active.
tx.frequency_hz	integer	Current TX centre frequency.
tx.sample_rate_sps	integer	Configured TX sample rate.
tx.power_dbm	number/null	[T3] Current TX power setting in dBm. null if set_tx_power not implemented.

Behaviour

- All fields listed as null when not available SHALL be present in the response with a JSON null value. They SHALL NOT be absent. This allows clients to parse unconditionally.
- The hardware_monitors object SHALL always be present at Tier 2 and above, even if all channels are null. The object itself is never null.
- The controls object reflects the current applied state, not the last requested state. If AGC is active and the hardware has autonomously changed rf_gain, controls.rf_gain reflects the hardware's current gain.
- actual_sample_rate_sps is particularly important for RTL-SDR hardware, which cannot achieve some requested rates exactly. A requested rate of 1,000,000 SPS may be achieved as 1,008,000 SPS. The client must use the actual rate for frequency scale calculations.

Example

```
{ "type": "request", "id": "st-1", "command": "get_status", "params": {} }

{ "type": "response", "id": "st-1", "status": "ok", "result": {
  "rx": {
    "enabled": true,
    "streaming": true,
    "frequency_hz": 14100000,
    "sample_rate_sps": 2048000,
    "actual_sample_rate_sps": 2048000,
    "reference_level_dbm": -30.0,
    "adc_overload": false,
    "adc_level_dbfs": -18.4,
    "lo_locked": true,
    "controls": {
      "rf_gain": 20.0, "agc": false, "bias_tee": 0.0,
      "ppm_correction": 0, "direct_sampling": "off"
    },
    "hardware_monitors": {
      "pa_temperature_celsius": null,
      "pa_supply_voltage_v": null,
      "pa_drain_current_ma": null
    }
  },
  "tx": {
    "enabled": false, "streaming": false,
    "frequency_hz": 14200000,
    "sample_rate_sps": 48000,
    "power_dbm": null
  }
}
```

TIP

At Tier 1, return unknown_command. At Tier 2, the hardware_monitors object with all nulls is a valid response for RX-only hardware. You do not need any PA sensors to implement get_status at Tier 2 — just return all three PA channels as null.

10.2 hardware_monitors Object

The hardware_monitors object provides a named-channel monitoring framework for hardware sensors. Three channels are defined as standard:

Channel name	Type	Description
pa_temperature_celsius	number/null	PA die or heatsink temperature in °C. null if no PA or no temperature sensor.
pa_supply_voltage_v	number/null	PA supply rail voltage in volts. null if not measurable.
pa_drain_current_ma	number/null	PA drain current in milliamperes. null if not measurable.

Additional channels from the reserved names list may be added in future revisions. Clients SHALL ignore any channel name they do not recognise.

11. Configuration Commands

11.1 set_frequency [T1-RX]

Required at: all profiles

Purpose

Set the centre frequency for either the RX or TX path. The device tunes the synthesiser as close to the requested frequency as the hardware permits and returns the actual applied frequency, which may differ slightly due to PLL step size. RX and TX frequencies are completely independent.

Request Fields

Field	Type	Required	Description
direction	string	YES	"rx" or "tx". Identifies which path to tune.
frequency_hz	integer	YES	Target centre frequency in Hz. Must be a 64-bit unsigned integer. Must fall within the ranges in capabilities.[direction].frequency_ranges_hz.

Response Fields (result object)

Field	Type	Description
direction	string	Echoed from request.
applied_frequency_hz	integer	The frequency actually applied by the hardware. May differ from the requested frequency due to synthesiser step size.

Behaviour

- The device SHALL apply the frequency immediately. If a stream is active, the frequency change takes effect without interrupting the stream.
- For direction=tx on an RX-only device (or a Tier 1 RX device), return not_supported.
- The device MAY emit a frequency_changed event after the synthesiser has settled. This is optional at Tier 1.

Error Codes

Error code	Condition
unsupported_frequency	Requested frequency is outside all ranges in capabilities.[direction].frequency_ranges_hz.
invalid_parameter	frequency_hz is zero, negative, or not an integer.
missing_parameter	direction or frequency_hz is absent.
not_supported	direction=tx on an RX-only device.

Example

```
// Tune RX to 14.1 MHz
{ "type": "request", "id": "f-1", "command": "set_frequency",
  "params": { "direction": "rx", "frequency_hz": 14100000 } }
{ "type": "response", "id": "f-1", "status": "ok",
  "result": { "direction": "rx", "applied frequency_hz": 14100000 } }
```

TIP

Store the current frequency for each direction in a variable. When `set_frequency` arrives, set the synthesiser (e.g. SI5351 over I2C) and store the result. The `applied_frequency_hz` in the response should be what the SI5351 actually achieved, not what was requested.

11.2 set_sample_rate [T1-RX]

Required at: all profiles

Purpose

Set the I/Q sample rate for either the RX or TX path. Only rates present in `capabilities.[direction].sample_rates_sps` are accepted. RX and TX rates are independent.

Request Fields

Field	Type	Required	Description
direction	string	YES	"rx" or "tx".
sample_rate_sps	integer	YES	Target sample rate in samples per second. Must be in the list from <code>capabilities.[direction].sample_rates_sps</code> .

Response Fields (result object)

Field	Type	Description
direction	string	Echoed.
applied_sample_rate_sps	integer	Rate actually applied. For RTL-SDR hardware this may differ from the requested rate due to clock divider constraints.

Behaviour

- The device SHALL only accept rates from its published `sample_rates_sps` list. Return `unsupported_sample_rate` for any other value.
- Changing the sample rate while a stream is active may require the stream to be stopped and reopened. If so, return `stream_reconfiguration_required`.
- The Tier 2 `get_status` response should reflect `actual_sample_rate_sps` if the achieved rate differs from the requested rate.

Error Codes

Error code	Condition
<code>unsupported_sample_rate</code>	Rate is not in <code>capabilities.[direction].sample_rates_sps</code> .
<code>stream_reconfiguration_required</code>	The rate change requires stopping and reopening the active stream.
<code>not_supported</code>	<code>direction=tx</code> on RX-only device.

Example

```
{ "type": "request", "id": "sr-1", "command": "set sample rate",
  "params": { "direction": "rx", "sample_rate_sps": 2048000 } }
{ "type": "response", "id": "sr-1", "status": "ok",
  "result": { "direction": "rx", "applied sample rate sps": 2048000 } }
```

11.3 set_control [T1-RX for rf_gain • T2 for all others]

rf_gain: all profiles | all others: Tier 2+

Purpose

Set a named hardware control to a new value. The set of available controls is reported by `get_capabilities.[direction].controls[]`. The type of value to send is determined by the control's type field in the descriptor.

Request Fields

Field	Type	Required	Description
direction	string	YES	"rx" or "tx".
name	string	YES	Control name as listed in <code>capabilities.[direction].controls[]</code> . E.g. "rf_gain", "agc", "bias_tee".
value	number/bool/string	YES	New value. Must match the control's type: number for float and integer, boolean for boolean, string for enum. For float controls with an <code>allowed_values</code> list, the value must be from that list.

Response Fields (result object)

Field	Type	Description
direction	string	Echoed.
name	string	Echoed control name.
applied_value	number/bool/string	The value actually applied by the hardware. For gain controls, this may be quantised to the nearest step.

Behaviour

- When a gain control (`rf_gain`, `lna_gain`, `if_gain`, or any vendor-defined gain control) changes, the device SHALL [T2] emit a VITA-49 Context packet with the updated Reference Level before the next IF Data packet. See §14 and SDR-SLC-VITA-1.0.
- When `agc` is set to true, the device ignores subsequent `rf_gain set_control` commands until `agc` is set to false. The device SHOULD emit `control_changed` events as the AGC adjusts the gain, and SHALL emit VITA-49 Context packets on each AGC gain change.
- When `agc` is set back to false, the device SHALL restore the last client-set `rf_gain` value.
- When `bias_tee` is set to a non-zero voltage, the device applies voltage to the antenna SMA connector. The default SHALL always be 0.0V on startup and after any TCP disconnect.
- When `direct_sampling` is changed, the device SHALL emit a VITA-49 Context packet with the updated Payload Format before the next IF Data packet.
- For float controls with an `allowed_values` array: the value must exactly match one of the listed values. Return `invalid_parameter` if it does not.

Error Codes

Error code	Condition
not_supported	Named control does not exist for this direction on this hardware.
invalid_parameter	Value is out of range, the wrong type, or (for float with allowed_values) not in the allowed list.
missing_parameter	direction, name, or value is absent.

Example

```
// Set gain (float)
{ "type":"request","id":"sc-1","command":"set_control",
  "params":{"direction":"rx","name":"rf_gain","value":24.0} }
{ "type":"response","id":"sc-1","status":"ok",
  "result":{"direction":"rx","name":"rf_gain","applied_value":24.0} }

// Enable AGC (boolean)
{ "type":"request","id":"sc-2","command":"set_control",
  "params":{"direction":"rx","name":"agc","value":true} }
{ "type":"response","id":"sc-2","status":"ok",
  "result":{"direction":"rx","name":"agc","applied_value":true} }

// Enable bias tee at 5V (float, allowed_values=[0.0,5.0])
{ "type":"request","id":"sc-3","command":"set_control",
  "params":{"direction":"rx","name":"bias_tee","value":5.0} }
{ "type":"response","id":"sc-3","status":"ok",
  "result":{"direction":"rx","name":"bias_tee","applied_value":5.0} }

// Set PPM correction (integer)
{ "type":"request","id":"sc-4","command":"set_control",
  "params":{"direction":"rx","name":"ppm_correction","value":-14} }

// Set direct sampling (enum)
{ "type":"request","id":"sc-5","command":"set_control",
  "params":{"direction":"rx","name":"direct_sampling","value":"i_branch"} }
```

TIP

At Tier 1 you only need to handle name=rf_gain. For any other name, return not_supported. The client will grey out controls that return not_supported.

11.4 get_control [T2]

Required at: Tier 2 and above

Purpose

Query the current applied value of a single named hardware control. Useful for refreshing the UI after a reconnect or for verifying that a set_control command took effect.

Request Fields

Field	Type	Required	Description
direction	string	YES	"rx" or "tx".
name	string	YES	Control name.

Response Fields (result object)

Field	Type	Description
direction	string	Echoed.

Field	Type	Description
name	string	Echoed.
value	number/bool/string	Current applied value. Type matches the control's type.

Error Codes

Error code	Condition
not_supported	Named control does not exist for this direction.

Example

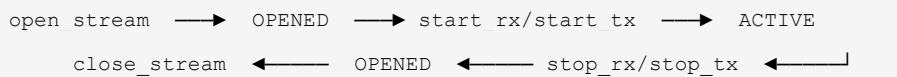
```
{ "type": "request", "id": "gc-1", "command": "get_control",  
  "params": { "direction": "rx", "name": "rf_gain" } }  
{ "type": "response", "id": "gc-1", "status": "ok",  
  "result": { "direction": "rx", "name": "rf_gain", "value": 24.0 } }
```

12. Stream Session Management [T1-RX]

A stream session represents an active or configured I/Q data flow. Every stream has a lifecycle: open → start → (active) → stop → close. The device assigns a `stream_id` when the stream is opened; this ID is used in all subsequent commands that refer to the stream.

On TCP disconnect, the device SHALL immediately stop all active streams and release all open stream sessions. Stream IDs SHALL NOT be reused within the same TCP session.

Stream lifecycle:



TCP disconnect: device auto-closes all sessions in any state.

12.1 open_stream (RX) [T1-RX]

Required at: all profiles

Purpose

Configure a VITA-49 RX stream session. The client specifies the UDP destination address and port to which the device will send VITA-49 packets. The device allocates a `stream_id` and returns the confirmed stream parameters including the source address and port.

Request Fields

Field	Type	Required	Description
direction	string	YES	"rx".
protocol	string	YES	Streaming protocol. Always "vita49" for current hardware.
protocol_version	string	YES	"VITA-A/1.0" for all current SDR-SLC hardware.
sample_rate_sps	integer	NO	Sample rate. If omitted, uses the last <code>set_sample_rate</code> value.
format	object	COND	I/Q format descriptor. If omitted, device uses the native:true format. See SDR-SLC-VITA-1.0 for format fields.
destination.ip	string	YES	Client IPv4 address. Device sends UDP packets here.
destination.port	integer	YES	Client UDP port to receive packets.

Response Fields (result object)

Field	Type	Description
stream_id	string	Opaque identifier for this stream. Use in <code>start_rx</code> , <code>stop_rx</code> , <code>close_stream</code> .
direction	string	"rx".

Field	Type	Description
source.ip	string	Device IP address. Packets will arrive from here.
source.port	integer	Device UDP source port for this stream.
sample_rate_sps	integer	Configured sample rate.
samples_per_packet	integer	Number of IQ pairs per UDP packet. Fixed for stream lifetime.
format	object	Confirmed I/Q format. Client must use this to decode packets.
vita.stream_id	integer	32-bit numeric Stream ID used in VITA-49 packet headers.
signal_type	string	[T2] "complex_iq", "real_i_branch", or "real_q_branch". Reflects current direct_sampling setting. Informational; Context packet is authoritative.

Behaviour

- Only one RX stream may be open at a time on single-channel hardware. Return `stream_already_open` if a second `open_stream(rx)` arrives.
- The device SHALL [T2] emit a VITA-A/1.0-CTX Context packet before the first IF Data packet after `start_rx`. This Context packet carries the signal type (complex/real) and current Reference Level.
- Clients that do not implement Context packet parsing SHALL silently discard VITA-49 packets with Packet Type 0x4.
- If no format is specified, the device uses the format with `native:true` as the default.

Error Codes

Error code	Condition
<code>stream_already_open</code>	An RX stream is already open. Call <code>close_stream</code> first.
<code>unsupported_stream_protocol</code>	Protocol name not in <code>capabilities.streaming_protocols</code> .
<code>unsupported_protocol_version</code>	Protocol version string not supported.
<code>unsupported_format</code>	Format not in <code>capabilities.rx.iq_formats</code> or <code>supported</code> is false.
<code>unsupported_sample_rate</code>	Rate not in <code>capabilities.rx.sample_rates_sps</code> .
<code>missing_parameter</code>	Required parameter absent.

Example

```
{ "type":"request","id":"os-1","command":"open_stream",
  "params":{
    "direction":"rx", "protocol":"vita49",
    "protocol version":"VITA-A/1.0",
    "format":{ "encoding":"unsigned","bits_per_component":8,
      "container_bits":8,"packing":"interleaved",
      "byte_order":"big_endian" },
    "destination":{"ip":"192.168.1.20", "port":60000 }
  } }

{ "type":"response","id":"os-1","status":"ok","result":{
  "stream_id"      : "rx-1",
  "direction"      : "rx",
  "source"         : { "ip":"192.168.5.35", "port":4621 },
  "destination"    : { "ip":"192.168.1.20", "port":60000 },
  "sample_rate_sps" : 2048000,
  "samples_per_packet": 360,
  "format"         : { "encoding":"unsigned","bits per component":8,
    "container_bits":8,"packing":"interleaved",
```

```

        "byte_order": "big_endian",
        "native": true, "supported": true, "label": "u8" },
    "vita"
      : { "stream_id": 1, "packet_class": "if_data",
        "timestamp mode": "integer and fractional" },
    "signal_type"
      : "complex_iq"
  } }

```

TIP

At Tier 1 you can ignore the format field in the request and always respond with your native format. Store the destination IP and port. You will need them when the VITA-49 UDP sender is configured.

12.2 open_stream (TX) [T1-RXTX]

Required at: Tier 1 RX/TX and above | Omit at: Tier 1 RX

Purpose

Configure a VITA-49 TX stream session. For TX the relationship is reversed: the client sends VITA-49 UDP packets TO the device. The client specifies its own source address in the request; the device returns the port it listens on for incoming TX packets.

Request Fields

Field	Type	Required	Description
direction	string	YES	"tx".
protocol	string	YES	"vita49".
protocol_version	string	YES	"VITA-A/1.0".
sample_rate_sps	integer	NO	TX sample rate.
format	object	COND	I/Q format for TX payload.
source.ip	string	YES	Client IP address. Packets will arrive from here.
source.port	integer	YES	Client UDP source port for TX packets.

Response Fields (result object)

Field	Type	Description
stream_id	string	Opaque TX stream identifier.
direction	string	"tx".
source.ip	string	Device IP. Not the device's TX listen address — see vita.
vita.stream_id	integer	32-bit Stream ID for VITA-49 packet headers.

Behaviour

- After start_tx, the client SHALL begin sending VITA-49 UDP packets to the device's TX listen port (capabilities.tx.vita_port) at the negotiated sample rate.
- If the client sends no packets, the device RF output is silence (zero I/Q).

Error Codes

Error code	Condition
not_supported	TX hardware not present.
stream_already_open	A TX stream is already open.

Example

```
{ "type": "request", "id": "os-tx", "command": "open stream",
  "params": {
    "direction": "tx", "protocol": "vita49",
    "protocol_version": "VITA-A/1.0",
    "sample_rate_sps": 48000,
    "format": { "encoding": "twos_complement", "bits_per_component": 16,
      "container_bits": 16, "packing": "interleaved",
      "byte_order": "big_endian" },
    "source": { "ip": "192.168.1.20", "port": 60001 }
  } }

{ "type": "response", "id": "os-tx", "status": "ok", "result": {
  "stream_id"      : "tx-1",
  "direction"      : "tx",
  "source"         : { "ip": "192.168.5.35", "port": 4622 },
  "destination"    : { "ip": "192.168.1.20", "port": 60001 },
  "sample_rate_sps" : 48000,
  "samples_per_packet": 360,
  "format"         : { "encoding": "twos_complement", "bits_per_component": 16,
    "container_bits": 16, "packing": "interleaved",
    "byte_order": "big_endian",
    "native": true, "supported": true, "label": "int16" },
  "vita"           : { "stream_id": 2, "packet_class": "if data",
    "timestamp_mode": "integer_and_fractional" }
} }
```

12.3 start_rx [T1-RX]

Required at: all profiles

Purpose

Begin transmitting VITA-49 UDP packets to the destination configured in open_stream(rx). The device starts the ADC and the UDP sender. The device emits a stream_state event when streaming begins.

Request Fields

Field	Type	Required	Description
stream_id	string	YES	Stream ID from open_stream response.

Response Fields (result object)

Field	Type	Description
direction	string	"rx".
streaming	boolean	true.

Behaviour

- The device SHALL emit a stream_state event with streaming=true and direction=rx immediately after (or alongside) the ok response.

- [T2] The device SHALL emit a VITA-A/1.0-CTX Context packet as the first UDP packet after start_rx, before any IF Data packets.

Error Codes

Error code	Condition
stream_not_found	stream_id not recognised.
stream_not_open	No RX stream is open.

Example

```
{ "type":"request","id":"rx-1","command":"start_rx","params":{"stream id":"rx-1" } }
{ "type":"response","id":"rx-1","status":"ok",
  "result":{" direction":"rx", "streaming":true } }
// Event:
{ "type":"event","event":"stream_state",
  "params":{" stream id":"rx-1","streaming":true,"direction":"rx" } }
// [T2] Context packet arrives as first UDP packet before IF Data packets.
```

12.4 stop_rx [T1-RX]

Required at: all profiles

Purpose

Stop transmitting VITA-49 UDP packets. The stream session remains open and can be restarted with start_rx without calling open_stream again.

Request Fields

Field	Type	Required	Description
stream_id	string	YES	Stream ID.

Response Fields (result object)

Field	Type	Description
direction	string	"rx".
streaming	boolean	false.

Behaviour

- The device SHALL emit a stream_state event with streaming=false and direction=rx.

Error Codes

Error code	Condition
stream_not_found	stream_id not recognised.

Example

```
{ "type":"request","id":"rx-2","command":"stop_rx","params":{"stream id":"rx-1" } }
{ "type":"response","id":"rx-2","status":"ok",
  "result":{" direction":"rx", "streaming":false } }
```

```
{ "type":"event","event":"stream_state",
  "params":{"stream_id":"rx-1","streaming":false,"direction":"rx" } }
```

12.5 start_tx [T1-RXTX]

Required at: Tier 1 RX/TX and above | Omit at: Tier 1 RX

Purpose

Begin accepting VITA-49 UDP packets from the client and outputting them to the DAC and PA. After start_tx, the client must send UDP packets at the negotiated sample rate to the device's TX VITA port.

Request Fields

Field	Type	Required	Description
stream_id	string	YES	TX Stream ID from open_stream(tx).

Response Fields (result object)

Field	Type	Description
direction	string	"tx".
streaming	boolean	true.

Behaviour

- The device emits a stream_state event with streaming=true and direction=tx.
- If turnaround_mode=auto, the device also automatically stops RX and emits a stream_state event for RX with streaming=false.
- If the client sends no UDP packets after start_tx, the RF output is silence.

Error Codes

Error code	Condition
not_supported	TX not available.
stream_not_found	TX stream ID not recognised.

Example

```
{ "type":"request","id":"tx-1","command":"start_tx","params":{"stream_id":"tx-1" } }
{ "type":"response","id":"tx-1","status":"ok",
  "result":{"direction":"tx","streaming":true } }
{ "type":"event","event":"stream_state",
  "params":{"stream_id":"tx-1","streaming":true,"direction":"tx" } }
```

12.6 stop_tx [T1-RXTX]

Required at: Tier 1 RX/TX and above

Purpose

Stop accepting TX UDP packets and turn off the PA. The TX stream session remains open.

Request Fields

Field	Type	Required	Description
stream_id	string	YES	TX Stream ID.

Response Fields (result object)

Field	Type	Description
direction	string	"tx".
streaming	boolean	false.

Behaviour

- The device emits stream_state with streaming=false, direction=tx. If turnaround_mode=auto, automatically resumes RX and emits stream_state for RX.

Error Codes

Error code	Condition
not_supported	TX not available.
stream_not_found	TX stream ID not recognised.

Example

```
{ "type":"request","id":"tx-2","command":"stop tx","params":{"stream id":"tx-1" } }
{ "type":"response","id":"tx-2","status":"ok",
  "result":{"direction":"tx","streaming":false } }
```

12.7 close_stream [T1-RX]

Required at: all profiles

Purpose

Release a stream session. The stream must be stopped (via stop_rx or stop_tx) before close_stream is called, or the device must stop it automatically. After close_stream, the stream_id is invalid.

Request Fields

Field	Type	Required	Description
stream_id	string	YES	Stream ID to close. Works for both RX and TX streams.

Response Fields (result object)

Field	Type	Description
closed	boolean	true.

Behaviour

- If the stream is still active when close_stream arrives, the device SHALL stop it first, then close it.
- The device SHALL release the stream_id and all associated resources.

Error Codes

Error code	Condition
stream_not_found	stream_id not recognised.

Example

```
{ "type":"request","id":"cs-1","command":"close_stream","params":{"stream_id":"rx-1" } }
{ "type":"response","id":"cs-1","status":"ok","result":{"closed":true } }
```

12.8 PTT and Turnaround Mode

Half-duplex transceivers must manage the transition between receiving and transmitting. The turnaround_mode field in get_capabilities tells the client which mode the device uses.

manual turnaround_mode

The client is responsible for stopping RX before starting TX, and restarting RX after stopping TX. This is the recommended mode for simple homebrew transceivers because it requires no state-machine logic on the device side.

```
// PTT press (manual mode):
C→S stop_rx(stream_id=rx-1)
S→C ok + event stream_state(rx-1, streaming=false)
C→S start_tx(stream_id=tx-1)
S→C ok + event stream_state(tx-1, streaming=true)
// Client sends UDP audio to device TX port.

// PTT release:
C→S stop_tx(stream_id=tx-1)
S→C ok + event stream_state(tx-1, streaming=false)
C→S start_rx(stream_id=rx-1)
S→C ok + event stream_state(rx-1, streaming=true)
```

auto turnaround_mode

The device has hardware T/R switching and manages RX muting internally. The client only calls start_tx and stop_tx. The device automatically mutes and unmutes RX and emits stream_state events for both directions.

```
// PTT press (auto mode):
C→S start_tx(stream id=tx-1)
S→C ok + event stream_state(tx-1, streaming=true)
S→C      event stream_state(rx-1, streaming=false) // auto-muted

// PTT release:
C→S stop_tx(stream id=tx-1)
```

```
S→C  ok + event stream_state(tx-1, streaming=false)
S→C      event stream_state(rx-1, streaming=true)    // auto-resumed
```

13. Transmitter Control Commands

The TX control commands beyond basic stream management are optional at Tier 1 RX/TX. They become required at Tier 3. For Tier 1 RX/TX, return `not_supported` for any command in this section that you have not implemented.

13.1 `get_tx_capabilities` [T1-RXTX]

Required at: Tier 1 RX/TX and above

Purpose

Return TX-specific capabilities in more detail than the top-level capabilities `tx` sub-object. The client calls this when the operator switches to TX mode. For a Tier 1 RX/TX device, the response can be minimal.

Response Fields (result object)

Field	Type	Description
<code>supported</code>	boolean	true if TX is available.
<code>frequency_ranges_hz</code>	array	TX frequency ranges.
<code>sample_rates_sps</code>	array	TX sample rates.
<code>iq_formats</code>	array	TX I/Q formats.
<code>vita_port</code>	integer	UDP port the device listens on for incoming TX packets.
<code>samples_per_packet</code>	integer	Expected IQ pairs per TX UDP packet.
<code>vswr_monitor</code>	boolean	[T3] true if <code>get_vswr</code> is available.
<code>auto_tune</code>	boolean	[T3] true if <code>auto_tune</code> is available.
<code>power_min_dbm</code>	number	[T2] Minimum output power in dBm. Omit if <code>set_tx_power</code> not implemented.
<code>power_max_dbm</code>	number	[T2] Maximum output power in dBm.
<code>power_step_dbm</code>	number	[T2] Power step size in dBm.

Behaviour

- A Tier 1 RX/TX device may set `vswr_monitor:false` and `auto_tune:false` and omit power fields.

Error Codes

Error code	Condition
<code>not_supported</code>	TX hardware not present.

Example

```
{ "type":"request","id":"tc-1","command":"get_tx_capabilities","params":{} }
{ "type":"response","id":"tc-1","status":"ok","result":{
  "supported" : true,
  "frequency_ranges_hz" : [{ "min":1800000, "max":30000000 }],
  "sample_rates_sps" : [48000, 96000, 192000],
```

```

"iq_formats"      : [{ "encoding":"twos_complement",
                        "bits_per_component":16,"container_bits":16,
                        "packing":"interleaved","byte_order":"big_endian",
                        "native":true,"supported":true,"label":"int16" }],
"vita_port"       : 4622,
"samples_per_packet" : 360,
"vswr_monitor"    : false,
"auto_tune"       : false
} }

```

TIP

For a simple Tier 1 RX/TX homebrew transceiver, this is all you need. No power control, no VSWR, no auto-tune.

13.2 set_tx_power [T2, optional at T1-RXTX]

SHOULD at Tier 1 RX/TX | Required at Tier 2+

Purpose

Set the PA output power in dBm. The device configures the PA gain or attenuation to achieve the requested power level.

Request Fields

Field	Type	Required	Description
power_dbm	number	YES	Target output power in dBm. Must be within power_min_dbm to power_max_dbm from get_tx_capabilities.

Response Fields (result object)

Field	Type	Description
applied_power_dbm	number	Power level actually applied.

Behaviour

- set_tx_power does not start transmission. It configures the level used on the next start_tx call.

Error Codes

Error code	Condition
unsupported_power	Power outside range.
not_supported	Power control not implemented.

Example

```

{ "type":"request","id":"tp-1","command":"set_tx_power","params":{"power_dbm":5.0 } }
{ "type":"response","id":"tp-1","status":"ok","result":{"applied_power_dbm":5 } }

```

13.3 get_vswr [T3]

Required at: Tier 3 | Return not_supported at T1 and T2**Purpose**

Query forward and reflected power and calculate VSWR. Only available when a directional coupler is fitted. Return not_supported if the hardware does not have a VSWR bridge.

Response Fields (result object)

Field	Type	Description
vswr	number	Standing wave ratio. 1.0 = perfect match.
forward_power_w	number	Forward power in watts.
reflected_power_w	number	Reflected power in watts.
return_loss_db	number	Return loss in dB.
measurement_frequency_hz	integer	Frequency at which measurement was taken.
valid	boolean	true if a TX session is active and measurement is valid.

Error Codes

Error code	Condition
not_supported	VSWR bridge not fitted.

Example

```
{ "type": "response", "id": "vs-1", "status": "ok", "result": {
  "vswr": 1.8, "forward_power_w": 5.0, "reflected_power_w": 0.2,
  "return_loss_db": 14.0,
  "measurement_frequency_hz": 14200000, "valid": true } }
```

13.4 auto_tune [T3]

Required at: Tier 3 | Return not_supported at T1 and T2**Purpose**

Trigger a mechanical or electronic antenna tuner cycle. The tuner adjusts L and C values to minimise reflected power at the specified frequency. This is always an operator-initiated action.

Request Fields

Field	Type	Required	Description
frequency_hz	integer	YES	Frequency at which to tune.
power_dbm	number	YES	Power level to use during tuning.

Response Fields (result object)

Field	Type	Description
tuned	boolean	true if target VSWR was achieved.
vswr_before	number	VSWR before tuning.
vswr_after	number	VSWR after tuning.
frequency_hz	integer	Echoed frequency.
tune_duration_ms	integer	Time the tune cycle took in milliseconds.

Behaviour

- The device SHALL NOT initiate a tune cycle automatically on TX session start or frequency change. This is a user-initiated command only.
- During tuning the device transmits a carrier. The operator must ensure the antenna is connected.

Error Codes

Error code	Condition
not_supported	Auto-tuner not fitted.
tune_failed	Tuner could not achieve the target VSWR. details.best_vswr contains the achieved value.

Example

```
{ "type": "request", "id": "at-1", "command": "auto_tune",  
  "params": { "frequency_hz": 14200000, "power_dbm": 5 } }  
{ "type": "response", "id": "at-1", "status": "ok", "result": {  
  "tuned": true, "vswr_before": 4.5, "vswr_after": 1.2,  
  "frequency_hz": 14200000, "tune_duration_ms": 3200 } }
```

14. Reference Level and Power Display [T2]

The Reference Level is the single value that converts I/Q sample amplitude to absolute received power at the antenna port, in dBm. Without it, the client can display relative levels but not absolute power. With it, the waterfall S-meter becomes a calibrated instrument.

The Reference Level answers the question: if a full-scale I/Q sample were produced, what power at the antenna port would cause it? A value of -30.0 dBm means that a signal at -30 dBm on the antenna will produce samples at maximum amplitude (0 dBFS).

The client calculates received power using:

```
P_dBm = ReferenceLevel_dBm + 20 × log10( RMS_amplitude / full_scale )

full_scale:    32767  (int16)
               127    (u8)
               8388607 (int24)
               1.0    (float32)

RMS for complex I/Q:  sqrt( mean(I2 + Q2) )
RMS for real (direct): sqrt( mean(I2) ) or sqrt( mean(Q2) )

Example:
ReferenceLevel = -30.0 dBm
Measured RMS = 3277 counts (int16) = 3277 / 32767 = 0.1 of full scale
Level in dBFS = 20 × log10(0.1) = -20.0 dBFS
P dBm = -30.0 + (-20.0) = -50.0 dBm
```

The Reference Level is reported through three complementary mechanisms. A client should use the most authoritative source available:

Mechanism	Authority	When to use
capabilities.rx.reference_level_dbm_at_preferred_gain	Low — static	Read once at startup. Initial value for power display before stream starts.
get_status rx.reference_level_dbm	Medium — current	Reflects current gain. Poll after gain changes if not parsing Context packets.
VITA-49 Context packet Reference Level field	High — authoritative	In-band, real-time, zero latency. Updates on every gain change during streaming.

The Reference Level tracks gain changes automatically. When the operator increases rf_gain by 10 dB, the device reduces the Reference Level by 10 dB. Full-scale samples now correspond to a 10 dB weaker signal at the antenna, because the hardware is amplifying more.

WARNING

*Reference Level represents a nominal calibration. A raw RTL-SDR dongle without characterisation: ± 5 – 8 dB absolute accuracy. A device with per-frequency per-gain lookup tables: ± 1 – 2 dB. Client implementations **SHOULD** display an accuracy indicator. Do not present uncalibrated power readings as precise measurements.*

15. Clock and Location [T2 query · T3 set]

Accurate timestamps in VITA-49 packets depend on the device having a synchronised clock. The clock source is reported in `get_clock_status`. The timestamp quality directly affects spectrum analysis accuracy and any time-domain correlations the client performs.

15.1 Clock Sources

Source	Accuracy	Notes
PTPv2 grandmaster (GPS-locked)	< 1 μ s	Best. UTC-traceable. Recommended for any serious work.
PTPv2 grandmaster (non-GPS)	< 100 μ s	Good. Accuracy depends on grandmaster quality.
GPS receiver on device	< 1 μ s	Good. Requires on-device GPS module.
Manual (<code>set_clock</code>)	Client accuracy	Acceptable for logging. Not UTC-traceable.
Free-running (no sync)	~1 ms/day drift	Integer timestamps count seconds since stream start, not since the Unix epoch.

NOTE

When source is `free_running`, the VITA-49 Timestamp Integer field is a relative counter (seconds since the most recent `start_rx`), not a Unix timestamp. Clients **SHOULD** warn the operator when `get_clock_status` returns `synced:false`.

15.2 `get_clock_status` [T2]

```
{ "type":"request","id":"ck-1","command":"get_clock_status","params":{} }
{ "type":"response","id":"ck-1","status":"ok","result":{
  "source"      : "ptpv2",
  "synced"      : true,
  "accuracy_us" : 5.2,
  "utc_time"    : "2026-05-20T14:32:01.000Z",
  "epoch_s"     : 1747751521,
  "ptp"         : { "grandmaster_id":"2c:cf:67:ff:fe:32:e6:54",
                    "clock_class":6, "available":true },
  "gps"         : { "available":false }
}
```

15.3 Location [T3]

`get_location` and `set_location` manage the operator's Maidenhead locator. Maidenhead SHALL use 6-character sub-square format with lowercase sub-square characters (e.g. JO22ki, not JO22KI). This information is embedded in VITA-49 GPS Context packets when available.

16. Asynchronous Events

Events are device-generated messages that the client receives without issuing a request. They notify the client of hardware state changes: stream status, frequency changes, gain adjustments, ADC overload, hardware monitor alerts. Clients SHALL silently ignore any event name they do not recognise.

A Tier 1 device only needs to emit `stream_state`. Everything else is optional at Tier 1 and required only at higher tiers as shown in the table.

Event name	Tier	Trigger	Key parameters
<code>stream_state</code>	[T1-RX]	Stream started, stopped, or failed.	<code>stream_id</code> , <code>streaming</code> (bool), <code>direction</code>
<code>frequency_changed</code>	[T2]	Synthesiser has settled to a new frequency.	<code>direction</code> , <code>frequency_hz</code>
<code>sample_rate_changed</code>	[T2]	Sample rate changed.	<code>direction</code> , <code>sample_rate_sps</code> , <code>actual_sample_rate_sps</code>
<code>control_changed</code>	[T2]	A control value changed (including AGC-driven changes).	<code>direction</code> , <code>name</code> , <code>value</code>
<code>adc_overload</code>	[T2]	ADC clipping detected. Emitted on onset, not continuously.	<code>stream_id</code> , <code>adc_level_dbfs</code> , <code>suggested_action</code>
<code>vswr_alert</code>	[T3]	VSWR exceeded threshold during TX.	<code>vswr</code> , <code>threshold</code> , <code>tx_reduced</code>
<code>monitor_alert</code>	[T3]	Hardware monitor channel crossed a threshold.	<code>channel</code> , <code>value</code> , <code>threshold</code> , <code>threshold_value</code> , <code>tx_inhibited</code>
<code>clock_sync_changed</code>	[T2]	PTP/GPS sync state changed.	<code>source</code> , <code>synced</code> , <code>accuracy_us</code>
<code>gps_fix_changed</code>	[T3]	GPS fix acquired or lost.	<code>fix</code> , <code>satellites</code> , <code>maidenhead</code>
<code>tuner_state</code>	[T3]	Antenna tuner cycle completed.	<code>tuned</code> , <code>vswr_after</code>
<code>warning</code>	[T2]	Non-fatal device condition.	<code>code</code> , <code>message</code>

16.1 `adc_overload` Event

This event is emitted when the ADC starts clipping. Clipping produces spurious signals on the waterfall that look like real signals. The client should display a prominent warning and recommend reducing gain.

```
{ "type": "event", "event": "adc_overload", "params": {
  "stream_id"      : "rx-1",
  "adc_level_dbfs" : -0.3,
  "suggested action": "reduce rf gain"
} }
```

16.2 monitor_alert Event

Emitted when a hardware_monitors channel crosses a warning or alarm threshold defined in capabilities.hardware_monitor_thresholds. The tx_inhibited field is true when the device has autonomously stopped TX output to protect the hardware.

```
// Warning threshold crossed:
{ "type":"event","event":"monitor_alert","params":{
  "channel"      : "pa_temperature_celsius",
  "value"        : 67.4,
  "threshold"    : "warning",
  "threshold value" : 65.0,
  "tx_inhibited" : false
} }

// Alarm threshold crossed (device stopped TX):
{ "type":"event","event":"monitor_alert","params":{
  "channel"      : "pa_temperature_celsius",
  "value"        : 81.2,
  "threshold"    : "alarm",
  "threshold value" : 80.0,
  "tx_inhibited" : true
} }
```

WARNING

When tx_inhibited is true in a monitor_alert, the device has autonomously stopped TX output. The client SHALL reflect this in the UI and SHALL NOT attempt start_tx until the condition clears and the device indicates it is safe to resume.

17. Error Code Registry

The following error codes are defined. Every error response SHALL use one of these codes in the error.code field. The error.message field is a human-readable string for display and logging.

Error code	When to use
unknown_command	Command name is not recognised. Return this for any command not in your implementation.
invalid_request	Malformed JSON, or a non-request message received where a request was expected.
missing_parameter	A required field is absent from params.
invalid_parameter	A field is present but has an invalid value or the wrong type. Include the field name in error.message.
unsupported_protocol_version	No version in the client's hello list is supported.
unsupported_frequency	Frequency outside all ranges in capabilities.[direction].frequency_ranges_hz.
unsupported_sample_rate	Rate not in capabilities.[direction].sample_rates_sps.
unsupported_format	I/Q format not in capabilities.iq_formats, or supported is false.
unsupported_stream_protocol	Streaming protocol name not supported.
unsupported_power	TX power outside the range in get_tx_capabilities.
stream_not_found	stream_id does not match any open session.
stream_already_open	open_stream called when a stream for this direction is already open.
stream_not_open	start_rx/start_tx called when no stream is open for this direction.
stream_reconfiguration_required	The requested change requires closing and reopening the active stream.
tune_failed	auto_tune could not achieve the target VSWR. details.best_vswr contains the achieved value.
clock_sync_active	set_clock rejected because an active sync source (PTP or GPS) is present.
busy	Device temporarily busy with another operation. Client should retry after a short delay.
device_error	Hardware or DSP error. Include diagnostic codes in error.details if available.
not_supported	Feature is not implemented on this hardware. Use for TX commands on RX-only devices, and for controls not available on this hardware.
timeout	A hardware operation timed out.
internal_error	Unexpected firmware error. Reconnect recommended. Device may be in an undefined state.

18. Extensibility and Forward Compatibility

The SLC-CP protocol is designed to grow without breaking existing implementations. The following rules govern how extensions work:

Scenario	Correct behaviour
Client sends a field the device does not know	Device silently ignores the unknown field.
Device sends a field the client does not know	Client silently ignores the unknown field.
Client sends an unknown command	Device returns <code>unknown_command</code> . Connection stays open.
Client receives an unknown event	Client silently ignores it.
Client receives an unknown control type in <code>controls[]</code>	Client ignores that descriptor gracefully. Does not abort the session.
New controls added to <code>controls[]</code>	Clients that do not know a control simply do not send <code>set_control</code> for it. Device returns <code>not_supported</code> if they try.
New <code>iq_formats</code> entries added	Added with <code>supported:false</code> until firmware implements them. Clients check the <code>supported</code> flag before requesting.
New error codes added	Clients treat unrecognised codes as <code>internal_error</code> .
Protocol minor version increment (1.0 → 1.1)	Backward compatible. Adds optional fields, commands, events. Existing implementations continue to work.
Protocol major version increment (1.x → 2.0)	Breaking change. Requires new hello version negotiation.

18.1 Reserved Field Names

The following names are reserved for future protocol revisions. Do not use them for other purposes in vendor extensions.

Reserved name	Location	Intended future use
<code>noise_figure_db</code>	<code>capabilities.rx</code>	Receiver noise figure in dB at the antenna port.
<code>ip3_dbm</code>	<code>capabilities.rx</code>	Input third-order intercept point in dBm.
<code>sfdr_db</code>	<code>capabilities.rx</code>	Spurious-free dynamic range in dB.
<code>gain_compression_1db_dbm</code>	<code>capabilities.rx</code>	1 dB gain compression input power.
<code>phase_noise_dbc_hz</code>	<code>capabilities.rx</code>	LO phase noise at 1 kHz offset.
<code>filter_bandwidth_3db_hz</code>	<code>capabilities.rx</code>	IF filter 3 dB bandwidth.
<code>oscillator_error_ppm_estimated</code>	<code>get_status rx</code>	Device-estimated oscillator error in PPM.
<code>frequency_correction_source</code>	<code>get_status rx</code>	Source of PPM correction: manual / gps / factory / auto.

Reserved name	Location	Intended future use
tuner_temperature_celsius	get_status rx	Tuner IC die temperature.
lna_temperature_celsius	hardware_monitors	External LNA temperature.
board_temperature_celsius	hardware_monitors	Main PCB temperature.
supply_voltage_v	hardware_monitors	Main board supply voltage.
total_current_ma	hardware_monitors	Total board current consumption.

19. Conformance Requirements

This section provides normative checklists for each implementation profile. An implementation claiming a profile SHALL satisfy all requirements for that profile and all profiles it includes.

19.1 Tier 1 Minimal RX Server

Tier 1 Minimal RX — Required implementation checklist

Requirement
Implement TCP transport with 4-byte big-endian length-prefixed JSON framing.
Implement hello: validate supported_protocol_versions, return selected version.
Implement get_capabilities: return schema, schema_version, full_duplex, and a minimal rx sub-object with frequency_ranges_hz, sample_rates_sps, vita_port, samples_per_packet, iq_formats, and controls containing at least rf_gain.
Implement set_frequency(direction=rx).
Implement set_sample_rate(direction=rx).
Implement set_control for name=rf_gain.
Implement open_stream(direction=rx, protocol=vita49).
Implement start_rx, stop_rx.
Implement close_stream.
Return unknown_command for any unimplemented command.
Return not_supported for direction=tx on set_frequency, set_sample_rate, open_stream, start_tx, stop_tx, set_control, get_control.
Return not_supported for set_control with any name other than rf_gain (unless additional controls are voluntarily implemented).
Silently ignore unknown JSON fields in any received message.
Treat all _hz fields as 64-bit unsigned integers.
Tear down all active streams on TCP connection loss.
Not reuse Stream IDs within a TCP session.
Emit stream_state event with direction field on start_rx and stop_rx.

19.2 Tier 1 Minimal RX/TX Server

Tier 1 Minimal RX/TX — All Tier 1 RX requirements, plus:

Requirement
Include turnaround_mode ("manual" or "auto") in get_capabilities.
Include a tx sub-object in get_capabilities with supported:true, frequency_ranges_hz, sample_rates_sps, vita_port, samples_per_packet, iq_formats.
Implement get_tx_capabilities.

Requirement
Implement set_frequency(direction=tx).
Implement set_sample_rate(direction=tx).
Implement open_stream(direction=tx, protocol=vita49).
Implement start_tx, stop_tx.
Emit stream_state event with direction=tx on start_tx and stop_tx.
For auto_turnaround_mode: emit stream_state events for both directions on start_tx and stop_tx.
Return not_supported for: get_vswr, auto_tune, set_tx_power (unless voluntarily implemented).

19.3 Tier 2 Standard Server

Tier 2 Standard — All Tier 1 RX/TX requirements, plus:

Requirement
Implement ping.
Implement get_status with all rx fields including actual_sample_rate_sps, reference_level_dbm, adc_overload, adc_level_dbfs, lo_locked (null where hardware cannot supply). Include hardware_monitors object with all three standard channels.
Implement get_clock_status.
Include preferred_sample_rate_sps, preferred_gain_db, reference_level_dbm_at_preferred_gain in capabilities.rx.
Include type field in all control descriptors in capabilities.rx.controls[].
Implement standard controls: agc (boolean), bias_tee (float, volts, allowed_values), ppm_correction (integer), direct_sampling (enum).
Implement get_control.
Implement set_tx_power.
Emit VITA-A/1.0-CTX Context packet before first IF Data packet after start_rx.
Emit VITA-A/1.0-CTX Context packet before first IF Data packet after signal mode change (direct_sampling change).
Emit VITA-A/1.0-CTX Context packet before first IF Data packet after any gain control change.
Include signal_type field in open_stream response.
Implement asynchronous events: frequency_changed, sample_rate_changed (with actual_sample_rate_sps), control_changed, adc_overload, clock_sync_changed, warning.
Include tx.supported:false and tx.channels:0 in capabilities if TX is not supported. The tx sub-object SHALL be present.
Not use reserved field names for other purposes.

19.4 Tier 3 Full Server

Tier 3 Full — All Tier 2 requirements, plus:

Requirement
Implement <code>get_vswr</code> .
Implement <code>auto_tune</code> .
Implement <code>set_clock</code> , <code>get_location</code> , <code>set_location</code> .
Implement <code>hardware_monitor_thresholds</code> in capabilities when threshold data is available.
Emit <code>monitor_alert</code> event when hardware monitor thresholds are crossed.
Set <code>tx_inhibited</code> correctly in <code>monitor_alert</code> when device has autonomously stopped TX.
Emit <code>vswr_alert</code> , <code>gps_fix_changed</code> , <code>tuner_state</code> events.
All response fields defined in this specification SHALL be present. Fields the hardware cannot supply SHALL be null, not absent.

19.5 Client Requirements (any tier)

Clients connecting to SDR-SLC devices SHALL:

Requirement
Send hello as the first message and verify <code>selected_protocol_version</code> before proceeding.
Call <code>get_capabilities</code> before any radio configuration command.
Treat all <code>_hz</code> fields as 64-bit unsigned integers. Use <code>BigInt</code> in JavaScript.
Silently ignore unknown JSON fields in all messages.
Silently ignore unknown event names.
Silently discard VITA-49 packets with Packet Type 0x4 (Context packets) if Context packet parsing is not implemented.
Not infer IF Data packet loss from Context packet Packet Count.
Handle <code>unknown_command</code> gracefully — disable the corresponding UI control rather than displaying an error to the operator.
Handle <code>not_supported</code> gracefully — grey out or hide the corresponding UI element.
Re-execute the full startup sequence (hello → <code>get_capabilities</code> → <code>configure</code> → <code>open_stream</code>) after any TCP disconnection.
Correlate responses by <code>id</code> field. Do not assume responses arrive in request order.
Use <code>actual_sample_rate_sps</code> from <code>get_status</code> (when non-null) for all frequency axis calculations.
Treat VITA-49 Context packet Reference Level as authoritative over SLC-CP <code>reference_level_dbm</code> .
Inhibit <code>start_tx</code> when <code>monitor_alert tx_inhibited</code> is true.

20. Security Considerations

SLC-CP version 1.0 is designed for trusted local area network operation. The protocol provides no authentication, authorisation, or encryption. Any host on the LAN that can reach TCP port 4620 can issue all control commands, including tuning the frequency, changing the gain, and starting transmission.

WARNING

Do not expose an SDR-SLC device on the Internet without a firewall. Any device that reaches port 4620 can start a TX stream on your amateur radio station. Network-level access controls (firewall rules, VLAN isolation) are the correct mitigation.

A future version of the protocol MAY introduce TLS 1.3 wrapping for the TCP connection, providing encryption and optional mutual certificate authentication. The length-prefixed framing is fully TLS-compatible without any protocol structure changes.

21. References

21.1 Normative References

Reference	Title	Relevance
SDR-SLC-PS-1.0	SDR-SLC Protocol Suite Architecture	Overall system architecture, network model, design rationale.
SDR-SLC-DISC-1.0	SLC-DISC Service Discovery	mDNS/DNS-SD advertisement. How clients find devices.
SDR-SLC-VITA-1.0	VITA-SLC I/Q Stream Transport	VITA-A/1.0 and VITA-A/1.0-CTX packet formats, timestamps, Reference Level encoding.
RFC 793	Transmission Control Protocol	TCP transport.
RFC 2119	Key Words for Use in RFCs	Normative keyword definitions (SHALL, SHOULD, MAY).
RFC 6762	Multicast DNS	mDNS protocol used by SLC-DISC.
RFC 6763	DNS-Based Service Discovery	DNS-SD used by SLC-DISC.
VITA-49.0	VITA Radio Transport Standard	Base VRT packet format that VITA-A/1.0 is a profile of.

21.2 Informative References

Reference	Title	Notes
RFC 4627	JSON Specification	JSON encoding rules.
IEEE 1588-2008	Precision Time Protocol v2	PTPv2 clock synchronisation (Tier 3 hardware).
IEEE 802 OUI	OUI Registry	IXChange BV OUI: 40-28-14.
RTL2832U datasheet	Realtek RTL2832U	Clock divider constraints relevant to <code>actual_sample_rate_sps</code> .
cJSON	Ultralight JSON parser	Recommended JSON library for STM32 / embedded implementations.
lwIP	Lightweight IP stack	TCP/IP stack for STM32H7 implementations.